



# Computer Science 3A

## MiniProject

2026-05-08

Deadline: 2026-05-08 12h00

Marks: 100

You, along with two to three others (4 people), must implement a practical project to demonstrate your proficiency with data structures discussed during this course (especially the Graph ADT) using the theme discussed below. The practical mini project must be submitted on or before the deadline (No Exceptions). Late submissions will not be accepted, and the student will receive zero for late assignments. This practical project will count as part of the practical component of this course (and 25% of your semester mark).

*Note: A signed plagiarism assignment submission form must be submitted alongside the practical assignment covering this assignment. The appropriate form will be available on Moodle, and a separate submission area will be available.*

The mini project demonstrates your understanding and proficiency with the concepts discussed in class and how they can be applied within a specific domain to solve a problem. You should use data structures we have not yet covered in class. In this case, you will need to research the implementation of the appropriate data structure. The practical mini project must meet the following requirements:

1. **Programming Language:** Java
2. **Submitted File Format:** Zip (Only!)
3. **Submitted File Naming Convention:** studentnumberminiproject.zip
4. The ZIP file must contain the following directory structure:
  - a. **src** - that contains all of the Java source files.
  - b. **dist** - that contains a executable jar file.
  - c. **ss** - that contains PowerPoint or PDF Slideshow for your Mini Project.
5. The use of third-party libraries for primary functionality is strictly prohibited; however, third-party libraries can be used for other functionality such as communications and visualisation (such as normal JavaFX, JFreeChart, GraphStream, JGraphT, Yworks, JUNG or JMonkey).
6. Your assignment must be executable from a jar file. If the assignment cannot be executed, you will receive zero.
7. If the assignment is too big to upload, please upload the source files to Moodle and contact the lecturers for the alternative upload method.
8. Your assignment must use a **graph-based structure** at its core.
9. The use of other data structures for auxiliary operations is encouraged (List, Stack,

---

Queue, Heap, Dictionary, Trees, etc.)

10. You must write the data structures yourself; you may use the textbook to guide you in implementing your data structures.
11. You may not do a practical implementation already assigned during the course.
12. Your project must be implemented as a desktop application based, with no data comms and APIs.
13. The assignment is a group project and should not overlap with the work of other groups or source code found on the Internet. Each assignment will be checked and if found guilty, will be sent up for disciplinary action.

## **Theme: Emergent Graph Structures for Image Similarity, Classification, and Path Finding**

In the era of big data and artificial intelligence, extracting meaningful patterns from unstructured data, such as images, remains a challenging problem. One promising approach is leveraging graph-based data structures, which excel at capturing complex relationships and contextual dependencies. The theme for this Mini Project is utilising emergent graph structures to facilitate image similarity detection, classification tasks, and pathfinding in images to solve a real-world problem in South Africa.

A mathematical graph is formally defined as “a set of nodes or points together with lines joining certain pairs of these points”<sup>1</sup>. In the context of images, these nodes represent individual pixels, regions, or feature descriptors, while the edges define relationships based on spatial proximity, feature similarity, or learned semantic connections. Graph-based representations allow us to model the structure within an image efficiently and relationships across multiple images in a dataset.

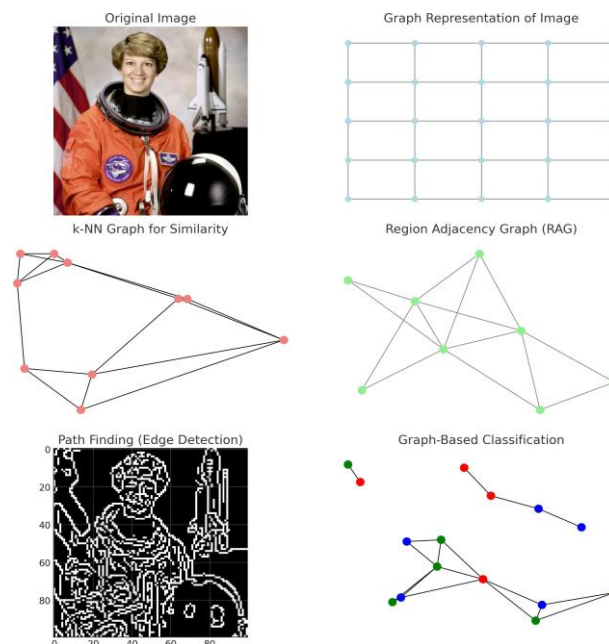
More specifically, in computer science, we work with graph abstract data types (ADT), which encapsulate possible values and operations on these values, making them useful for real-world problem-solving. When applied to images, graph-based structures can help solve the following:

1. Image segmentation and object detection
2. Feature extraction and similarity search
3. Classification using graph neural networks (GNNs)
4. Pathfinding in images (e.g., shortest path in navigation, robotic vision, or maze solving)

---

<sup>1</sup> 1Bondy, J. A., & Murty, U. S. R. (1976). Graph theory with applications (Vol. 290). London: Macmillan.

A visual representation of how graphs can model images for classification is shown in the figure below:



Graph-based image analysis techniques offer advantages over traditional methods by preserving spatial relationships and capturing hierarchical dependencies. Some key applications of graph structures in image processing include:

1. **Region Adjacency Graphs (RAGs):** Used for superpixel segmentation and hierarchical clustering.
2. **Graph Neural Networks (GNNs):** Applied to learn embeddings for classification tasks.
3. **k-Nearest Neighbor Graphs (k-NN):** Used for content-based image retrieval and similarity measures.
4. **Minimum Spanning Trees (MST):** Utilized for edge detection and contour tracing.
5. **Spectral Graph Theory:** Leveraged in image compression and feature representation.
6. **Graph-Based Path Finding:** Algorithms such as Dijkstra's or A\* can be applied to find optimal paths in an image-based graph (e.g., the shortest route through an environment or maze-solving tasks).

This mini project aims to develop a system that utilises graph-based techniques to analyse images, focusing on similarity detection, classification, and/or path finding. The implementation should:

- Construct a graph representation of images using an appropriate method (e.g., k-NN, RAG, MST, or path-finding graphs).
- Implement **two (2)** of the following graph-based tasks:
  - **Similarity detection**
  - **Classification**
  - **Path finding**
- Develop a user interface that allows users to interact with the system, apply it to multiple images, and visualize results.

Your practical implementation must demonstrate the use of graph-based image processing techniques as the primary computational approach. The system should explore image relationships, enable robust classification, or provide effective path-finding solutions using emergent graph structures.

Examples of mini projects you may NOT implement include (i.e., the ban list that will result in you getting zero):

1. **A utility library** where there is no user interface (remember, we want to interact with it and see it work).
2. **Anything copied from the Internet or a previous project**, e.g., simple edge detection without novel graph-based enhancements. This constitutes plagiarism, and appropriate disciplinary action will follow (don't do it).

Marks will be awarded based on the scope of your practical mini-project, the depth of your graph-based implementation, the effectiveness of the user interface, and the final presentation. Guidance will be provided throughout the semester to ensure your project meets these objectives.

You must confirm your project by **27th of March 2026** at noon. The method of confirming the project topic will be a Google Form (<https://forms.gle/jt3u5aXcBbMHptxE7>) and the outcome (out of 5, where two and below is a rejection of your project) will be shown on Moodle for each student's topic.

Peer evaluation will be conducted through structured reviews, where each participant will assess another group's implementation based on clarity, functionality, and effectiveness. Feedback should include constructive criticism and suggestions for improvement, ensuring a fair and comprehensive assessment of the system's performance.

## Marksheet

- |                                                                                                                                                                            |      |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------|------|
| 1. <b>Abstraction</b> (Successfully translates problem domain to the correct constructs in a graph)                                                                        | [10] |
| 2. <b>Use of a Graph to solve the problem</b> (CRUD of nodes and edges)                                                                                                    | [20] |
| 3. <b>Logic and Complexity</b> (Can facilitate solution processes using graph algorithms, provides a dynamic graph and systematically solves the problem)                  | [30] |
| 4. <b>Novelty</b> (New or unusual problem choice, attributes, etc.)                                                                                                        | [10] |
| 5. <b>Look and Feel</b> (Aesthetics - A graphical user interface that facilitates the use of the graph)                                                                    | [20] |
| 6. <b>Video with Slideshow</b> (An 8-minute video describing their project with a slideshow that depicts all the necessary aspects and provides screenshots for processes) | [10] |